

## **Trabajo Práctico Integrador**

### **"Optimización de Infraestructura mediante la Virtualización de un Servidor de Servicios Integrados"**

#### **Alumnos**

Manchini, Maria Virginia

Martinez, Braian Ivan

**Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.**

**Arquitectura y Sistemas Operativos**

#### **Docente Tutor**

Sanchez, Santos

## Índice

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Introducción.....                                                       | 3  |
| Objetivos .....                                                         | 3  |
| Marco Teórico .....                                                     | 4  |
| Caso Práctico.....                                                      | 5  |
| - SECCIÓN I: Preparación del Entorno y Creación de la VM.....           | 5  |
| - SECCIÓN II: Instalación del Sistema Operativo Invitado.....           | 8  |
| - SECCIÓN III: Primer Inicio y Verificación de Servicios.....           | 9  |
| - SECCIÓN IV: Automatización y Sincronización con Repositorio Remoto... | 10 |
| Metodología Utilizada .....                                             | 13 |
| Resultados Obtenidos .....                                              | 14 |
| Conclusiones.....                                                       | 15 |
| Bibliografía .....                                                      | 16 |

## Introducción

El presente trabajo integrador aborda la virtualización de hardware como una solución estratégica para la gestión eficiente de la infraestructura tecnológica en el entorno empresarial.

La elección de este tema se basa en la necesidad de comprender como las empresas modernas resuelven la subutilización de sus recursos de computación.

Frente a la problemática de una empresa que mantiene tres servidores físicos independientes (Web, Base de Datos y Archivos), la virtualización surge como la respuesta técnica óptima para consolidar dichos servicios dentro de un único Servidor Virtual con Ubuntu Server, el cual se ejecuta sobre un hardware anfitrión moderno. De esta manera, se reducen costos operativos, se maximiza el rendimiento del procesador y se centraliza la administración de las aplicaciones en un solo entorno virtualizado

Para un técnico en programación, comprender la arquitectura de sistemas operativos y la virtualización es crucial. Aprender a configurar y dimensionar entornos virtuales nos permite diseñar aplicaciones mas eficientes, entender la comunicación entre sistemas y simular laboratorios de desarrollo y pruebas locales idénticos a los entornos de producción reales.

## Objetivos

- Diseñar un escenario de consolidación de servicios empresariales mediante el uso de Oracle VirtualBox en una única instancia virtual
- Configurar de manera lógica los recursos (CPU, RAM, almacenamiento) para un sistema operativo invitado (Ubuntu Server 24.04 LTS) encargado de alojar los servicios de la organización.
- Demostrar las ventajas de la abstracción de hardware en la administración de sistemas operativos modernos.

## Marco Teórico

- A. Virtualización: es la tecnología que permite crear versiones virtuales de recursos informáticos, dividiendo el hardware real en múltiples plataformas lógicas independientes llamadas Maquinas Virtuales (VM). Cada VM cuenta con sus propios recursos asignados y ejecuta su propio SO invitado de manera aislada.
- B. Hipervisor: es un software que permite crear y gestionar maquinas virtuales (VMs). Actúa como intermediario entre el hardware físico y los sistemas operativos que se ejecutan en la VMs.  
Existen dos tipos:
- 1) *Tipo 1 (Bare-metal)*: se instala directamente sobre el hardware, sin necesidad de un sistema operativo anfitrión. Ej.: Microsoft Hyper-V.
  - 2) *Tipo 2 (Hosted)*: se instala como una aplicación dentro de un sistema operativo anfitrión. Ej.: VirtualBox. Este es el modelo utilizado en el presente trabajo.
- C. Redes virtuales: La comunicación dentro de un entorno virtualizado requiere estructurar canales de red específicos para aislar o exponer servicios:
- ✓ NAT: Las VMs comparten la IP del anfitrión, lo que simplifica el acceso a internet
  - ✓ Bridge Networking: Cada VM recibe su propia IP en la red física, ideal para entornos de prueba avanzados.
  - ✓ Red interna: Exclusiva para comunicación entre VMs dentro del mismo hypervisor
- D. Snapshots: son capturas del estado completo de una VM en un momento dado, incluyendo la memoria, el estado del CPU y el disco. Son fundamentales en entornos de prueba.

## Caso Práctico

Una empresa cuenta con tres computadoras físicas de escritorio obsoletas, dedicadas exclusivamente a un único servicio cada una:

1. Servidos Web: aloja el sitio institucional y la plataforma de clientes.
2. Servidor de Base de Datos: almacena los registros de usuarios y operaciones.
3. Servidor de Archivos: guarda los documentos compartidos de la oficina.

Esta infraestructura física genera altos costos de electricidad, ocupa espacio innecesario y el hardware de cada equipo esta utilizado a menos de su capacidad real.

Solución propuesta: adquirir un único servidor físico de prestaciones modernas y consolidar los tres entornos mediante un único servidor virtual independiente (Ubuntu Server) ejecutado en VirtualBox.

## SECCIÓN I: Preparación del Entorno y Creación de la VM

- 1) *Preparación del Entorno:* Antes de iniciar con el despliegue de la máquina virtual, se procedió con la instalación y preparación del software hipervisor en el sistema anfitrión.

- Verificación inicial: "Virtualización Activa"

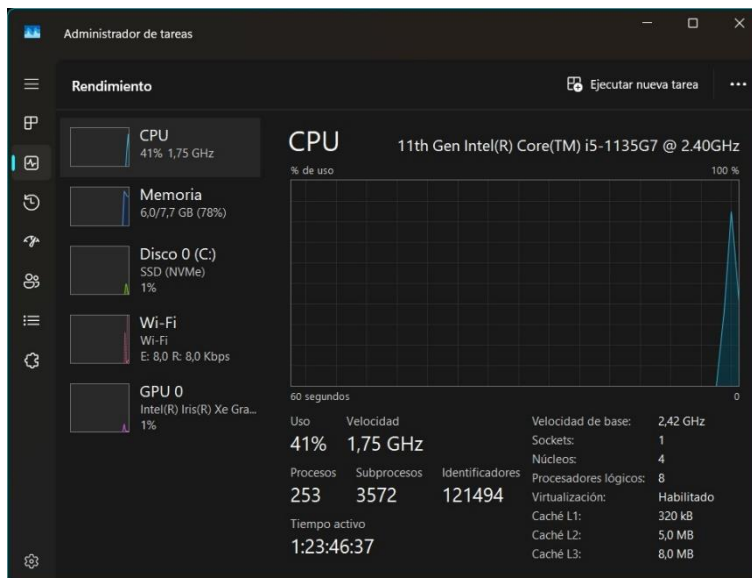


Figura 1: Verificación de soporte de virtualización por hardware activo en el sistema operativo anfitrión.

- Instalar VirtualBox

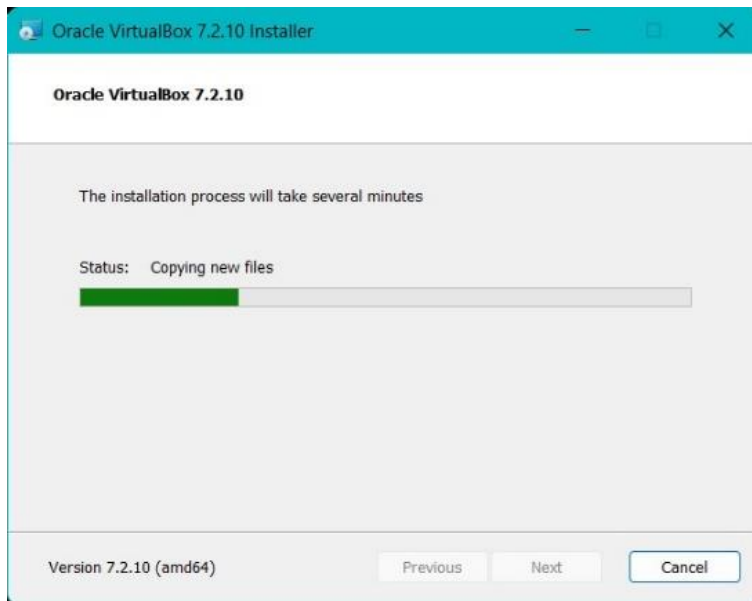


Figura 2: Proceso de instalación de las librerías y componentes del hipervisor Oracle VirtualBox en el equipo local.

- 2) *Creación de la Máquina Virtual:* Se creó la máquina virtual denominada "UbuntuServer-TPI" utilizando Oracle VirtualBox. Durante esta etapa se seleccionó la imagen ISO de Ubuntu Server 24.04 LTS y se configuró la instalación manual sin unattended installation.

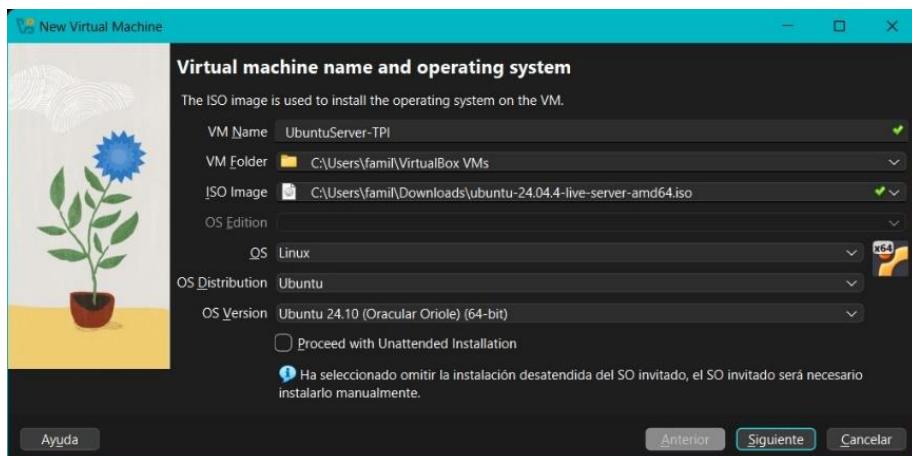


Figura 3: Configuración inicial del nombre de la VM, asignación de la ISO de Ubuntu Server y desactivación de la instalación desatendida.

3) *Configuración de Hardware*: Se asignaron los recursos de hardware de la máquina virtual:

- Memoria RAM: 4096 MB (4 GB)
- Procesadores: 2 CPU
- Disco duro virtual: 25 GB
- Firmware: BIOS (EFI deshabilitado)

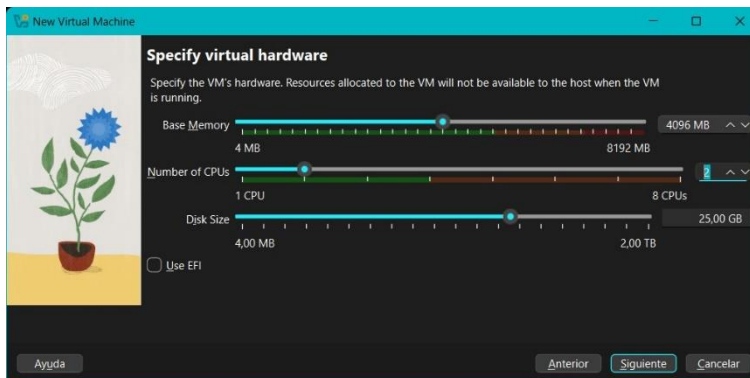


Figura 4: Asignación de recursos lógicos (memoria RAM, núcleos de CPU y tamaño total del almacenamiento masivo).

4) *Resumen y Creación de la Máquina Virtual*: Se validó la configuración de red de la máquina virtual utilizando el Adaptador 1 en modo NAT. Esta configuración permite acceso a Internet desde la máquina virtual utilizando la conexión del equipo anfitrión sin requerir configuración adicional de red.

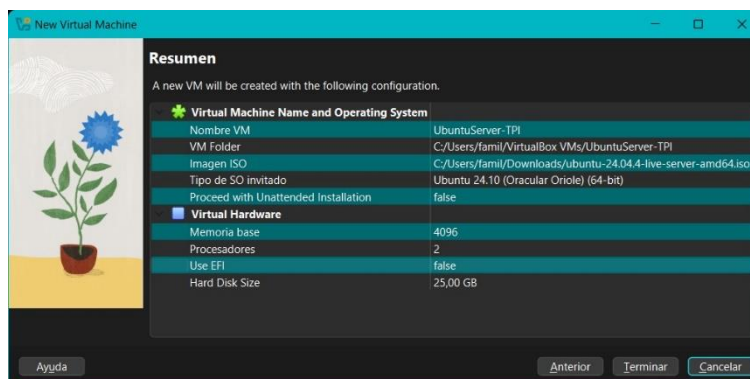


Figura 5: Pantalla de resumen con las especificaciones técnicas globales de la máquina virtual antes de su confirmación.

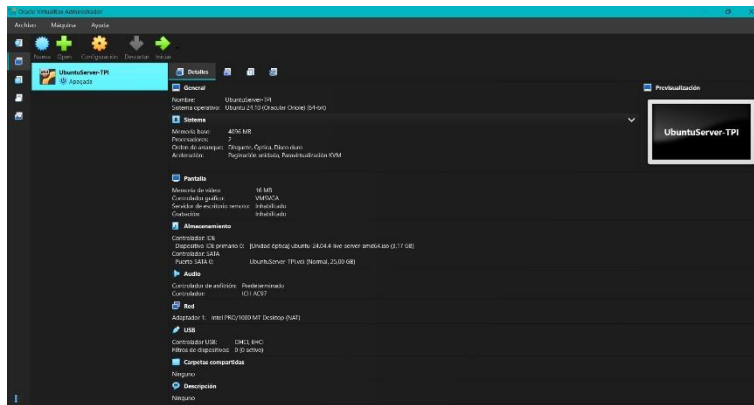


Figura 6: Máquina virtual creada (VM).

## SECCIÓN II: Instalación del Sistema Operativo Invitado

- 5) *Estructura de Almacenamiento Lógico*: Se inició el proceso de instalación de Ubuntu Server 24.04 LTS desde la imagen ISO previamente configurada. Se establecieron el idioma, distribución de teclado y parámetros regionales. El instalador detectó de forma automática la interfaz de red enp0s3 y asignó dirección IP mediante DHCP dentro de la red NAT. Finalmente, se seleccionó el uso completo del disco virtual de 25 GB utilizando la configuración automática propuesta por Ubuntu Server.

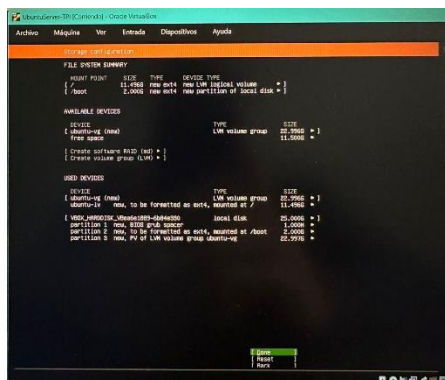


Figura 7: Esquema de particionado guiado del disco de 25 GB aplicando sistemas de archivos Ext4 y volúmenes lógicos (LVM).

- 6) *Parámetros de Identidad del Servidor*: Se configuró el usuario principal del sistema (braian) y el nombre del servidor (srv-virtualizacion). Asimismo, se seleccionó la instalación del servicio OpenSSH Server para permitir futuras conexiones remotas y administración del sistema. Se completó exitosamente la instalación de Ubuntu Server y se reinició la máquina virtual.



## SECCIÓN III: Primer Inicio y Verificación de Servicios

- 7) *Primer Inicio y Login:* Se verificó el acceso al sistema mediante autenticación del usuario configurado durante la instalación en la interfaz de línea de comandos (TTY1).

```
ubuntu 24.04.4 LTS srv-virtualizacion tty1
srv-virtualizacion login: braian
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri Jun 19 01:26:33 PM UTC 2025
System load:      0.13
Usage of /:        42.3% of 11.21GB
Memory usage:     2%
Swap usage:       0%
Processes:        119
Users logged in:   0
IPV4 address for enp0s3: 10.0.2.15
IPV6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fee0:9784
Expanded Security Maintenance for Applications is not enabled.

29 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

braian@srv-virtualizacion:~$
```

Figura 8: Consola de Ubuntu Server tras el arranque inicial, demostrando la existencia del usuario "braian" mediante el prompt de inicio de sesión exitoso.

- 8) *Verificación de Identidad y Red:* Se validaron correctamente los parámetros vitales del sistema operativo e infraestructura de red virtualizada a través de comandos nativos de la consola (hostname e ip a):

- Hostname del servidor: srv-virtualizacion
- Dirección IP asignada: 10.0.2.15 en la interfaz enp0s3
- Conectividad de red y Versión del sistema operativo.

```
braian@srv-virtualizacion:~$ hostname
srv-virtualizacion
braian@srv-virtualizacion:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e0:97:84 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 85201sec preferred_lft 85201sec
    inet6 fd17:625c:f037:2:a00:27ff:fee0:9784/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86232sec preferred_lft 14232sec
    inet6 fe80::a00:27ff:fee0:9784/64 scope link
        valid_lft forever preferred_lft forever
braian@srv-virtualizacion:~$
```

Figura 9: Ejecución de comandos de diagnóstico de red e identidad que confirman el hostname "srv-virtualizacion" y el direccionamiento IP activo.

- 9) *Entorno de Desarrollo Python*: Se comprobó la disponibilidad de Python 3.12.3 dentro del servidor Ubuntu mediante el parámetro `--version`. Seguidamente, se generó el directorio local de trabajo destinado al proyecto mediante el comando `mkdir TPI_Virtualización` y se validó su ruta absoluta en el almacenamiento con `pwd`.

```
braian@srv-virtualizacion:~$ python3 --version
Python 3.12.3
braian@srv-virtualizacion:~$ mkdir TPI_Virtualizacion
braian@srv-virtualizacion:~$ cd TPI_Virtualizacion
braian@srv-virtualizacion:~/TPI_Virtualizacion$ pwd
/home/braian/TPI_Virtualizacion
braian@srv-virtualizacion:~/TPI_Virtualizacion$
```

Figura 10: Auditoría del intérprete de Python instalado y creación del entorno local del proyecto.

## SECCIÓN IV: Automatización y Sincronización con Repositorio Remoto

- 10) *Script de Diagnóstico del Sistema*: Se desarrolló el archivo `diagnostico.py` dentro de Ubuntu Server utilizando un editor de texto por consola. El código fuente importa módulos del sistema (`socket`, `platform`, `datetime`, `sys`, `getpass`) para automatizar el chequeo de variables críticas de arquitectura de computadoras.

```
diagnostico.py
Script de diagnóstico del entorno virtualizado.

import socket
import platform
import getpass
import sys
from datetime import datetime

def main():
    """Muestra información básica del entorno virtualizado."""
    servidor = socket.gethostname()
    usuario = getpass.getuser()
    fecha_hora = datetime.now()

    print("\n" * 40)
    print("TPI - Virtualización")
    print("\n" * 40)

    print(f"Servidor: {servidor}")
    print(f"Usuario: {usuario}")
    print(f"Fecha y hora: {fecha_hora.strftime('%d/%m/%Y %H:%M:%S')}")
    print(f"Sistema Operativo: {platform.system()} {platform.release()}")
    print(f"Versión de Python: {sys.version}")

    ip = socket.gethostbyname(socket.gethostname())
    print(f"Dirección IP: {ip}")
    print("Estado de Red: OK")

    print()
    print("Entorno virtualizado operativo.")
    print("Proyecto académico - Arquitectura y Sistemas Operativos.")
    print("\n" * 40)

if __name__ == "__main__":
    main()

braian@srv-virtualizacion:~/TPI_Virtualizacion$
```

Figura 11: Código fuente desarrollado en Python para la lectura automatizada de variables del sistema operativo.

- 11) *Validación de Ejecución del Script*: Se verificó el funcionamiento del programa mediante su ejecución exitosa en consola invocado con python3 diagnostico.py, retornando de forma limpia todos los metadatos del hardware virtualizado.

```
from datetime import datetime
def main():
    """Muestra información básica del entorno virtualizado."""
    servidor = socket.gethostname()
    usuario = getpass.getuser()
    fecha_hora = datetime.now()
    print("%s" % 40)
    print("TPI - Virtualización")
    print("%s" % 40)
    print(f"Servidor: {servidor}")
    print(f"Usuario: {usuario}")
    print(f"Fecha y hora: {fecha_hora.strftime('%d/%m/%Y %H:%M:%S')}")
    print(f"Sistema Operativo: {platform.system()} {platform.release()}")
    print(f"Versión de Python: {sys.version}")
    ip = socket.gethostbyname(socket.gethostname())
    print(f"Dirección IP: {ip}")
    print("Estado de Red: Ok")
    print()
    print("Entorno virtualizado operativo.")
    print("Proyecto académico - Arquitectura y Sistemas Operativos.")
    print("%s" % 40)

brain@srv-virtualizacion:~/TPI-Virtualizacion$ python3 diagnostico.py
TPI - Virtualización
Servidor: brain
Usuario: brain
Fecha y hora: 22/06/2026 14:02:00
Sistema Operativo: Linux 5.0.8-124-generic
Versión de Python: 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0]
Dirección IP: 127.0.0.1
Estado de Red: Ok
Entorno virtualizado operativo.
Proyecto académico - Arquitectura y Sistemas Operativos.
```

Figura 12: Ejecución del script en la terminal de Ubuntu Server y salida de datos con la información del entorno virtual.

- 12) *Gestión de Código y Control de Versiones*: Se creó el repositorio remoto "TPI-Virtualización" en GitHub para centralizar el código fuente, documentación y evidencias. Se estructuraron las carpetas organizativas del repositorio (script\_python, capturas, diagramas, documentación) mediante archivos. gitkeep y se registró el primer commit. Finalmente, se incorporó el archivo diagnostico.py, registrando el segundo commit y sincronizando mediante un Push Origin hacia la nube.

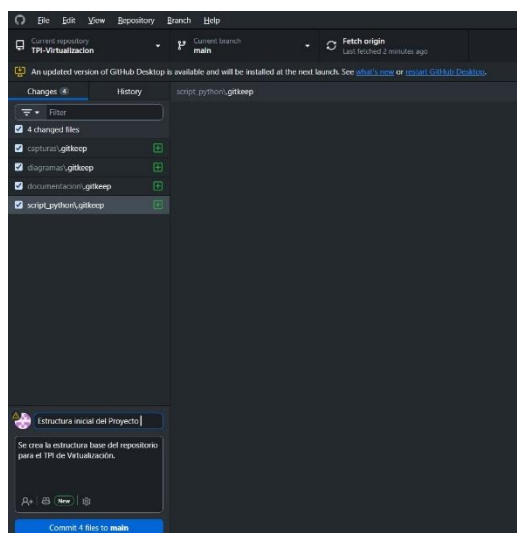


Figura 13: Interfaz de control de versiones (GitHub Desktop) documentando el árbol de carpetas y el registro del primer Commit a la rama principal (main).

13) *Gestión del Trabajo Colaborativo y Revisión de Código:* Con el fin de simular un entorno de desarrollo profesional y asegurar la integridad de la documentación en la rama principal (main), se implementó un flujo de trabajo basado en Pull Requests (Solicitudes de extracción).

La integrante del equipo (Virginia-10) realizó modificaciones sobre el archivo README.md en una rama secundaria y solicitó la incorporación de los cambios. El administrador del repositorio (Bramart96) verificó el código, aprobó la revisión y procedió a la unificación de los archivos.

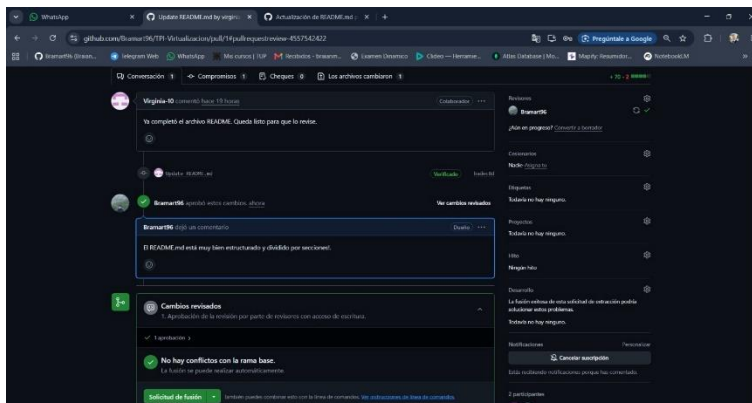


Figura 14: Interfaz mostrando la revisión por pares (Code Review), comentarios de aprobación y estado listo para la fusión de ramas.

14) *Historial de Commits Consolidado:* Tras la aprobación de la solicitud de extracción, se ejecutó la fusión (Merge) de la rama de documentación de forma exitosa. Se realizó una auditoría final sobre el historial (Commits) del repositorio remoto para comprobar la correcta trazabilidad de los aportes de cada integrante y la integración de los diagramas, scripts y textos del proyecto.

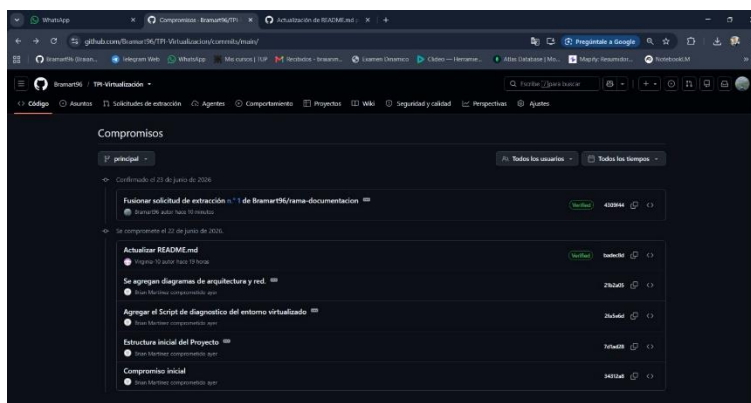


Figura 15: Historial unificado de commits, que evidencia el flujo de trabajo colaborativo.

## Metodología Utilizada

Para el desarrollo del presente trabajo integrador, el equipo distribuyó las tareas de forma colaborativa, dividiendo las actividades en: implementación técnica en laboratorio y confección de la documentación académica.

El proceso secuencial se estructuró de la siguiente manera:

Investigación preliminar: Se consultó la documentación oficial de Oracle VirtualBox y las guías de despliegue de Ubuntu Server 24.04 LTS para establecer los requisitos mínimos de hardware del entorno.

Diseño y asignación de recursos: Se determinó un esquema de dimensionamiento lógico para la máquina virtual (4 GB de memoria RAM, 2 CPUs y 25 GB de almacenamiento virtual) con el fin de garantizar el funcionamiento fluido del sistema operativo invitado sin comprometer la estabilidad del sistema anfitrión.

Implementación del laboratorio: Se procedió con la creación de la máquina virtual en VirtualBox, la configuración de la red en modo NAT y la instalación manual del sistema operativo. Posteriormente, se verificó el entorno local mediante comandos de diagnóstico de red y de versiones de software (Python 3).

Gestión de código y versionado: Se creó un repositorio remoto en GitHub para centralizar la estructura del proyecto. Se desarrolló un script de diagnóstico en Python (diagnostico.py) dentro del entorno virtualizado, validando su ejecución local antes de realizar la sincronización final mediante comandos Git (commit y push).

### Trabajo Colaborativo

Integrante 1 – Martínez, Braian Ivan

- Instalación y configuración de VirtualBox.
- Creación de la máquina virtual Ubuntu Server.
- Configuración de red y servicios.
- Instalación de OpenSSH.
- Desarrollo del script Python de diagnóstico.
- Creación y administración del repositorio GitHub.
- Registro de commits y control de versiones.
- Generación de evidencias técnicas.
- Elaboración de diagramas y material gráfico.

Integrante 2 – Manchini, Maria Virginia

- Elaboración de documentación académica.
- Redacción del informe final.
- Organización de evidencias.
- Revisión final de la documentación.

## Resultados Obtenidos

El desarrollo del caso práctico permitió alcanzar un entorno virtualizado completamente funcional y correctamente documentado.

A través de este proyecto se logró la creación e inicio de la máquina virtual denominada “UbuntuServer-TPI” utilizando un hipervisor de Tipo 2 (VirtualBox). El SO invitado detecto correctamente la interfaz de red (enp0s3), obteniendo una dirección IP de forma dinámica (10.0.2.15) mediante el servicio DHCP provisto por la red NAT de VirtualBox, garantizando el acceso seguro a internet. Se comprobó la operatividad del entorno Unix y la disponibilidad nativa de Python 3.12.3. el script de diagnóstico diseñado se ejecutó de forma óptima, imprimiendo por consola las variables de entorno, el hostname (srv-virtualización) y el estado de red. La integración con GitHub Desktop y la terminal Git facilito el resguardo de la estructura del proyecto de manera organizada y transparente. Durante la instalación se omitió la instalación desatendida (unattended installation) en VirtualBox para asegurar una configuración manual de las particiones de almacenamiento y de las credenciales de usuario administrador.

## Conclusiones

La realización de este trabajo integrador permite concluir que la virtualización de hardware es una competencia fundamental en la formación de un Técnico Universitario en Programación. Mediante el uso de un hipervisor hospedado, se demostró la capacidad de abstraer los recursos físicos de una computadora para dar vida a un entorno de servidor aislado, seguro y eficiente.

La consolidación de servicios dentro de una única instancia virtualizada de Ubuntu Server evidenció un excelente balance en el uso de recursos lógicos, esta arquitectura permite a las organizaciones optimizar costos de hardware y mantenimiento.

Finalmente, la integración del laboratorio virtual con herramientas de control de versiones como GitHub consolida una metodología de trabajo idéntica a los escenarios de producción reales en la industria del software.

## Bibliografía

- ✓ **Oracle Corporation.** (2026). *Oracle VM VirtualBox®: User Manual*. Recuperado de <https://www.virtualbox.org/manual/>
- ✓ **Canonical Ltd.** (2026). *Ubuntu Server Guide: Documentation*. Recuperado de <https://ubuntu.com/server/docs>
- ✓ **Silberschatz, A., Galvin, P. B., & Gagne, G.** (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons. Capítulo 16: "Virtual Machine".
- ✓ **Stallings, W.** (2023). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
- ✓ Universidad Tecnológica Nacional. (2026). *¿Qué es la virtualización y como funciona?. Hipervisor Tipo 1 y Tipo 2. Apunte Virtualización de recursos*. Cátedra de Arquitectura y Sistemas Operativos. Recuperado del campus virtual institucional.